

Project Proposal: SocialOps — Multi-Agent Social Media Command Center

1. Domain

Digital marketing / social media management for small and mid-size businesses. A domain every team member can speak to and interviewers immediately understand.

2. Problem Statement

Small businesses running multiple brand accounts face three recurring problems: incoming comments and mentions pile up unanswered (especially after a post spikes), content is produced inconsistently across platforms, and nobody has time to analyze what's working.

SocialOps addresses this with a multi-agent orchestration layer. An orchestrator receives each incoming item and routes it to specialist agents: a triage agent (classifies comments and scores sentiment), a response agent (drafts brand-voice replies), a content agent (turns one uploaded asset into platform-specific posts), a media agent (analyzes images/videos and checks brand guidelines), and an analytics agent (summarizes performance). A human approves anything before it publishes.

From a cloud computing standpoint, the application is deliberately designed as a stateful monolith first, then migrated step by step through the six-phase project flow so that each cloud concept (compute, containers, managed data services, decoupled messaging, orchestration, observability, automation) is exercised on a real workload.

3. Technical Fit & Tech Stack

Dimension	How SocialOps uses it	Front end	Application layer	Data store
Relational data	Accounts, campaigns, posts, comments, approvals, and per-agent run logs with FK relationships and audit history	React/Next.js dashboard, served from a container then from S3 + CloudFront	FastAPI (Python) REST API, containerized	PostgreSQL: local → on the VM → Amazon RDS (Multi-AZ)
Unstructured file processing	Uploaded product photos/videos processed by the media agent; bulk comment exports (CSV/JSON) parsed by the triage agent	Drag-and-drop upload	Worker containers calling Bedrock (vision + text) and Rekognition	Local disk → EBS → Amazon S3 with lifecycle policies
Async / traffic-spike	A viral post produces thousands of comments in an hour; ingestion is queued and fanned out to	Live queue-depth and agent-status panel	Orchestrator + specialist agent workers as separate containers behind SQS; horizontal	Amazon SQS for messaging, ElastiCache (Redis) for shared

Dimension	How SocialOps uses it	Front end	Application layer	Data store
	autoscaling agent workers with a dead-letter queue		autoscaling on queue depth	agent state and session cache

Mapping to the project flow

Phase	What we do	Cloud concepts exercised
1. Working Local App	Monolithic FastAPI + React app with local Postgres, local file storage, and agents running in-process via Docker Compose	Baseline architecture, local dev parity
2. Platform Setup & Lift-and-Shift	AWS account, IAM roles, VPC with public/private subnets, security groups; deploy the monolith as-is to an EC2 instance with Postgres and files on the same box	IaaS, networking, IAM, least privilege
3. Containerize & Deploy	Dockerize API, front end, and agent workers; push to ECR; run on ECS (Fargate) behind an Application Load Balancer	Containers, image registry, managed container runtime, load balancing
4. Decouple State	Move Postgres to RDS, files to S3, agent state and sessions to ElastiCache, work items to SQS; app containers become stateless	Managed data services, object storage, caching, message queues, stateless design
5. Orchestrate & Expose	Split into services (API, orchestrator, triage workers, media workers); autoscale workers on SQS queue depth; expose via API Gateway + ALB with HTTPS, Route 53, and WAF; agent hand-offs coordinated by Step Functions	Service decomposition, autoscaling, API management, edge security, workflow orchestration
6. Observe & Automate	CloudWatch dashboards, alarms, and structured logging; X-Ray tracing across agents; Terraform for all infrastructure; GitHub Actions CI/CD to build, test, and deploy; k6 load tests; cost dashboard	Observability, IaC, CI/CD, load testing, FinOps

4. Target User

A marketing manager at a small business, or a boutique agency account manager running 3–5 client brand accounts. Not a developer; they live in a dashboard, review agent output, and approve work.

Resume value: high. The project demonstrates a full cloud migration path (lift-and-shift → containers → managed services → orchestration → automation) on top of multi-agent orchestration and unstructured media processing, which is exactly the story a cloud or platform engineering interviewer wants to hear.

5. Desired Outcomes

Once fully deployed, SocialOps should absorb engagement spikes with no human intervention, cut time-to-first-response on customer comments from hours to minutes, and let one marketing manager run multiple accounts with agents doing the first pass. Architecturally, the system should be fully stateless at the compute layer, scale out and in automatically on queue depth, survive an AZ failure, and be redeployable from scratch through IaC and CI/CD.

6. Definition of Success

Technical: the same application runs correctly at every phase of the flow; after Phase 4 the app containers hold no state; replaying a 2,000-comment viral-post dump triggers visible worker scale-out and the queue drains within a target window with failures landing in the DLQ; any team member can destroy and recreate the environment with `terraform apply`; a CI pipeline deploys on merge.

Demo: upload one product photo and watch three platform-specific drafts appear; replay the comment dump and show the SQS depth spike, ECS task count rising, and CloudWatch recovery graph; approve a batch of agent-drafted replies; kill a container live and show traffic keep flowing; show the before/after architecture diagrams from Phase 1 to Phase 6.

Known risk: live social platform APIs (X, Meta, TikTok) require paid access or lengthy app review. We will build against a replay/synthetic data source and treat live integration as a stretch goal.